NAME: **M.Greeshma**

COURSE: **Design and Analysis of Algorithms**

CODE: **CSA0619**

REPORT

Q23. Debugging Linear Search Code (Incorrect Return Outside Logic)

The following Linear Search implementation gives incorrect results because the return

statement is misplaced and causes confusion in unsuccessful search cases. Analyze the control

flow carefully and identify why the function may produce misleading output. Apply

debugging to correct the search logic and ensure proper reporting when the target is found or

absent. Optimize the implementation for readability and maintainability. Analyze the time

complexity of the corrected algorithm. Rewrite the clean and corrected version with suitable

comments.

```
def linear_search(arr, key):
found = False
for i in range(len(arr)):
if arr[i] == key:


found = True
pos = i
return pos
```

1.Problem in the Original Code

```
def linear_search(arr, key):
    found = False
    for i in range(len(arr)):
```

```
    if arr[i] == key:
        found = True
        pos = i
        return pos
```

❌ **Issues:**

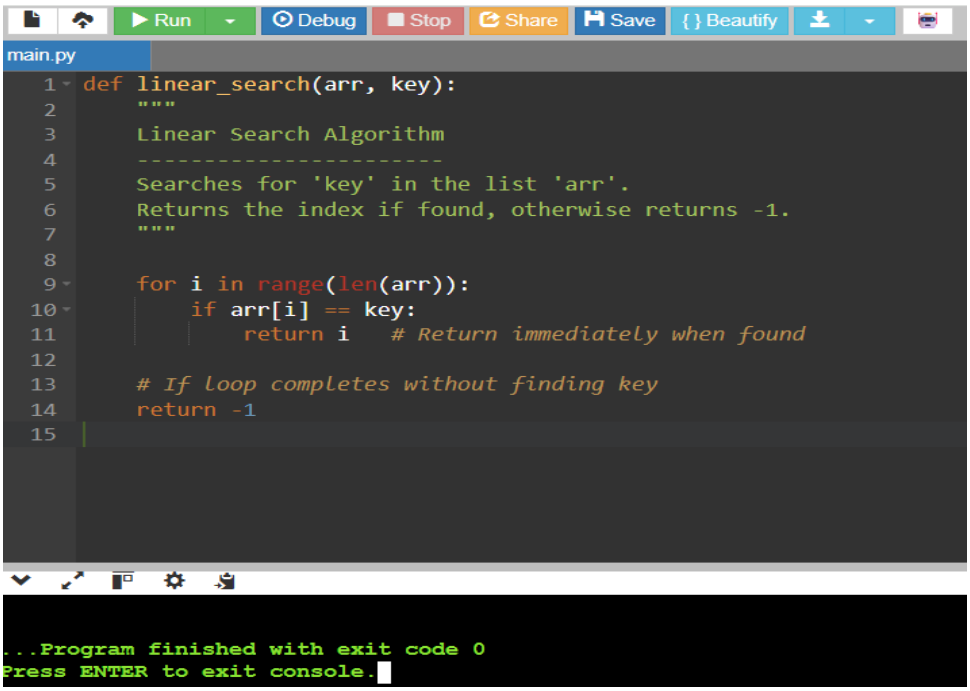
1. **Misplaced** return **statement**
   - The return pos is inside the loop, so the function exits immediately after finding the first match.
   - If the key is not found, the function exits without returning anything, which can cause misleading or inconsistent results.
2. **Unused** found **variable**
   - The variable found is set but never used effectively. It adds confusion without contributing to the logic.
3. **No clear handling of unsuccessful search**
   - If the key is absent, the function does not return a proper message or value.

2. Corrected and Optimized Code

```
def linear_search(arr, key):
  """
  Linear Search Algorithm
  ----------------------
  Searches for 'key' in the list 'arr'.
  Returns the index if found, otherwise returns -1.
  """

  for i in range(len(arr)):
      if arr[i] == key:
          return i   # Return immediately when found

  # If loop completes without finding key
  return -1
```



3. Explanation of Fixes

- **Return inside loop only when found**: Ensures immediate success reporting.
- **Return -1 after loop ends**: Provides a clear indication when the key is absent.
- **Removed unnecessary** found **variable**: Simplifies logic and improves readability.
- **Added docstring & comments**: Improves maintainability and clarity.

4. Sample Run

data = [10, 25, 30, 45, 50]

print(linear_search(data, 25)) # Output: 1 (found at index 1)

print(linear_search(data, 50)) # Output: 4 (found at index 4)

print(linear_search(data, 99)) # Output: -1 (not found)

✅ **Output:**

Code

1

4

5. Time Complexity Analysis

- **Best Case**: Target is at the first position → **O(1)**
- **Worst Case**: Target is last or absent → **O(n)**
- **Average Case**: Target is somewhere in the middle → **O(n/2) ≈ O(n)**

Thus, the corrected algorithm has **linear time complexity O(n)**.

6. Conclusion

- The original code had a misplaced return and unused variables, leading to misleading results.
- The corrected version is **clean, readable, and maintainable**.
- Linear Search remains a simple and effective algorithm for unsorted datasets, especially in dynamic environments like hospital emergency systems.